



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KINH TẾ - LUẬT

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF ECONOMICS AND LAW

DATA MINING

Lecture 3: Characterization and Comparison



Lecture 3 outline

Section	Syllabus item	What we cover
3.5.	Prepares item 3	Measuring data for classification: entropy and information gain, chi-square, correlation, discretization
3.6.	Items 1 and 2	Data generalization and summarization-based characterization: concept hierarchies, data cube against attribute-oriented induction
3.7.	Items 3 and 4	Analytical characterization and class comparison: target and contrast classes, t-weight and d-weight, characteristic and discriminant rules
3.8.	Item 6	Association rule mining, correlation analysis and constraint-based mining: support, confidence, Apriori, lift, constraints
Lab.	—	Lab 3 in Google Colab on the crawled data

Where we left off: Lecture 2 on one page

2.1 The four attribute types

- Nominal, binary, ordinal, numeric; numeric splits into interval and ratio
- The type decides what you may compute
- store_id is nominal: the average of two ids means nothing

Nominal	Binary	Ordinal	Numeric
Color {Red, Blue, Green}	Has card {Yes, No}	Rank {1, 2, 3, ...}	Age {...}

Numeric splits

Interval



Temperature
(-10°C, 10°C, ...)

Ratio



Income
(0, 10M, 20M, ...)



The attribute type determines which operations are possible (e.g., average, range, order, etc.).

2.2 Describing one attribute

- Centre: mean, median, mode; their order shows the skew
- Spread: range, quartiles, IQR, standard deviation
- Basket: mode 45,000 < median 97,000 < mean 108,856
- The boxplot comes from the five-number summary, and $1.5 \times \text{IQR}$ flags candidates

Example: Income (numeric)



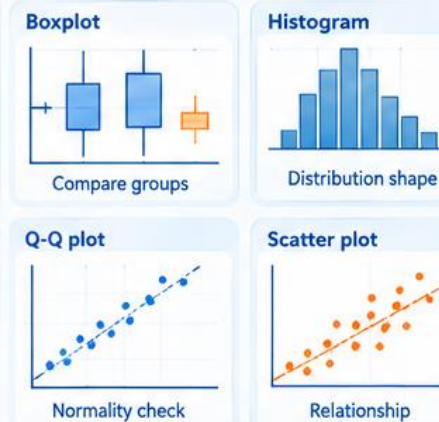
Other examples



2.3 Choosing the chart

- Boxplot compares groups, histogram shows the true shape
- Q-Q plot checks normality, scatter shows a relationship
- Every chart hides something: bin width and axis range are choices

Different charts, different questions



Choose the right chart for the right question.

2.4 Measuring proximity

- Nominal: simple matching, asymmetric binary: Jaccard
- Numeric: Minkowski, as Manhattan, Euclidean or supremum
- Ordinal: rank, then map to [0, 1]; cosine ignores size
- The attributes you feed in decide who counts as similar

Types of measures

Nominal
Simple matching
e.g., {A, B, C} = {A, B, C}

Binary (asymmetric)
Jaccard
e.g., {A, B} vs {A, C}

Numeric
Minkowski / Manhattan / Euclidean
e.g., $(x_1, x_2, \dots, x_n)$

Ordinal
Rank $\rightarrow [0, 1]$
e.g., cosine (ignores size)

Similarity depends on the attribute type and the method.



The big picture:
We start with data ...



2.1
Understand the attributes
(what can we compute?)



2.2
Describe one attribute
(how does it look?)



2.3
Choose the right chart
(what does it tell us?)



2.4
Measure proximity
(how similar are they?)



... to prepare for finding patterns in the next steps.

From Describing Data to Discovering Patterns



2.5 MEASURE

Which attributes matter for the class?

Example

Which attribute best separates Young and Adult customers?

Age	Income	Student	Buys Laptop	Class
22	12M	Yes	Yes	Young
24	15M	Yes	Yes	Young
45	40M	No	No	Adult
51	50M	No	No	Adult
28	20M	Yes	Yes	Young
...	...	...	...	...

Methods

- Entropy**
How mixed are the classes?
- Information Gain**
How much less mixed after splitting?
- Chi-square**
Is the attribute related to the class?
- Correlation**
Do two numeric variables move together?
- Discretization**
Can we turn continuous values into useful groups?

2.6 SUMMARIZE

Can we make the data simpler?

Example

From detailed values to higher-level representations.

Raw data		Generalize	Simpler view	
Age	Income		Age group	Income group
22	12M	→	Young	Low
23	15M		Young	Medium
24	18M		Young	Medium
25	20M		Adult	Medium
26	22M		Adult	High
...	...		...	...

Concept hierarchy

```

    graph TD
      Location --> Vietnam
      Location --> Thailand
      Vietnam --> HCMC
      Vietnam --> Hanoi
      HCMC --> District 1
      HCMC --> District 2
  
```

Data cube

Concept hierarchy = zoom out one attribute.
Data cube = look at the data from several dimensions.

2.7 COMPARE

How are the classes different?

Example

What describes Young customers?
What separates Young from Adult customers?

Young (Target class)	Adult (Contrast class)
• Student = Yes (80%)	• Student = No (95%)
• Income < 20M (75%)	• Income > 30M (85%)
• Age < 30 (90%)	• Age > 40 (90%)

Characteristic rules	Discriminant rules
What is common in the target class?	What distinguishes the classes?

Young VS Adult

2.8 DISCOVER

What attributes occur together?

Example

Find items that frequently appear together in a transaction.

Shopping basket data

T1	{Bread, Milk, Butter}
T2	{Bread, Milk}
T3	{Bread, Eggs}
T4	{Milk, Butter}
...	...

Association rules

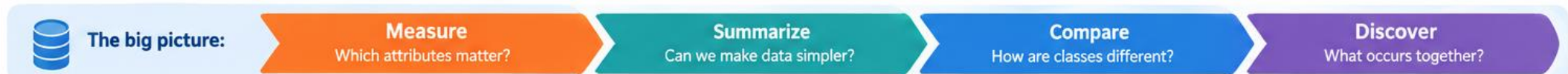
Bread → Milk

Support
How often do they appear together?

Confidence
When Bread occurs, how often does Milk occur?

Lift
Is the relationship stronger than random co-occurrence?

Also: Correlation analysis & constraint-based mining (Apriori)



3.5

Measuring Data for Classification

3.5 Measuring Data for Classification

From objects to classes

What changes when a class label exists

In 3.4 every row was equal: how far apart are two rows?

Now **one column is special**: the class label, often binary

The question moves to the columns: **which one separates the classes?**

A label is an attribute.

Three places the answer is used

- Analytical characterization (3.7): drop the attributes that carry no signal before generalizing
- Decision trees (Chapter 4): the attribute with the highest gain becomes the root split
- Feature selection in any model: fewer, better attributes and less noise

ONE RULE FOR THE WHOLE SECTION

Every measure here compares the class distribution before and after you know an attribute. If knowing the attribute leaves the class as mixed as before, the attribute is irrelevant, however impressive it looks.

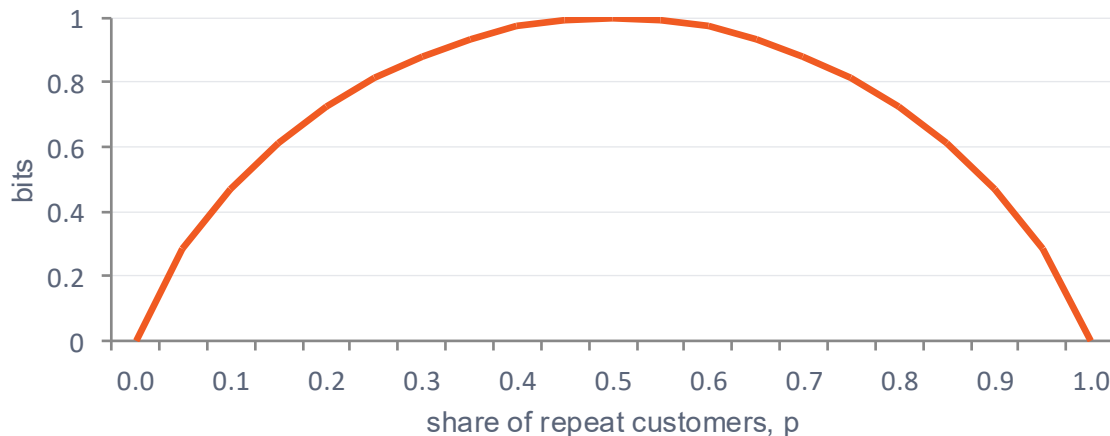
3.5 Measuring Data for Classification

Entropy: how mixed a class is

```
Info(D) = - \sum p_i \cdot \log_2(p_i) \quad i = 1 \dots m \text{ classes}
p_i = \text{share of class } i \text{ in the data set } D
```

```
14 customers: 9 repeat, 5 one-time
Info(D) = -(9/14) \cdot \log_2(9/14) - (5/14) \cdot \log_2(5/14)
        = 0.940 \text{ bits}
```

Entropy of a binary label against the share of the positive class



How to read the number

- 0 bits: every customer carries the same label, nothing left to learn
- 1 bit: a 50/50 split, the most uncertain a binary label can be
- 0.940 bits: 9 against 5 is almost as uncertain as a coin toss

WHY THE LOGARITHM IS BASE 2

The unit is the bit. One perfectly designed yes/no question halves the candidates, so entropy is the expected number of such questions. Pinning down a label with entropy 0.940 takes 0.94 of a question on average. The base only changes the unit; it never changes the ranking of attributes, which is what we are after.

3.5 Measuring Data for Classification

Information gain on fourteen Saigon Bean customers

customer	channel	loyalty_card	basket_band	store_type	repeat
C101	delivery	no	low	mall	no
C102	app	yes	high	campus	yes
C103	delivery	yes	low	mall	yes
C104	in-store	yes	mid	campus	yes
C105	app	yes	high	mall	yes
C106	app	yes	mid	mall	yes
C107	in-store	no	high	campus	yes
C108	delivery	no	mid	campus	no
C109	in-store	yes	low	mall	no
C110	app	yes	low	mall	yes
C111	in-store	no	high	campus	no
C112	app	no	mid	campus	yes
C113	in-store	yes	low	mall	yes
C114	in-store	no	mid	campus	no

$\text{Info}(D) = 0.940 \text{ bits}$ (9 yes, 5 no)

$\text{Info_channel}(D)$
= $6/14 \cdot \text{Info}(3, 3)$ in-store
+ $5/14 \cdot \text{Info}(5, 0)$ app
+ $3/14 \cdot \text{Info}(1, 2)$ delivery
= $6/14 \cdot 1.000 + 5/14 \cdot 0.000 + 3/14 \cdot 0.918$
= 0.625 bits

$\text{Gain}(\text{channel}) = 0.940 - 0.625 = 0.315 \text{ bits}$

Before looking at the channel, the label carried 0.940 bits of uncertainty. Split the customers by channel and 0.625 bits remain: the channel removed a third of what we did not know. App users are all repeat customers, and that pure group is where most of the gain comes from.

3.5 Measuring Data for Classification

Gain for every attribute, and the ranking

Attribute	Partition, repeat yes/no per value	Info_A(D)	Gain(A)	Verdict
channel	in-store 3/3 · app 5/0 · delivery 1/2	0.625	0.315	Most relevant
loyalty_card	yes 7/1 · no 2/4	0.704	0.236	Relevant
store_type	mall 5/2 · campus 4/3	0.924	0.016	Noise
basket_band	low 3/2 · mid 3/2 · high 3/1	0.925	0.015	Noise

WHAT THE RANKING BUYS YOU

Section 3.7 drops `store_type` and `basket_band` before generalizing, because 0.015 bits is what an attribute that knows nothing about the label produces.

A KNOWN BIAS

Gain favours attributes with many values. `customer_id` has 14 values, one customer each, and a gain of 0.940: perfect and useless. Gain ratio divides by the split information to correct this, and the Gini index used by CART is the other common choice. Both come back in Chapter 4.

3.5 Measuring Data for Classification

Chi-square: is a nominal attribute independent of the class?

	repeat	one-time	total
card = yes	150	50	200
card = no	90	210	300
total	240	260	500

```
expected = row total × column total / n
card yes, repeat    200 × 240 / 500 = 96
card yes, one-time  200 × 260 / 500 = 104
card no,  repeat    300 × 240 / 500 = 144
card no,  one-time  300 × 260 / 500 = 156
```

$$\begin{aligned}\chi^2 &= \sum (\text{observed} - \text{expected})^2 / \text{expected} \\ &= 54^2/96 + 54^2/104 + 54^2/144 + 54^2/156 \\ &= 30.4 + 28.0 + 20.3 + 18.7 = 97.4\end{aligned}$$

WHERE 3.841 COMES FROM

The chi-square table, 1 degree of freedom, 5% level. scipy skips the table and returns the p-value directly.

How to decide

- Null hypothesis: the card and repeat buying are independent
- Degrees of freedom = (rows – 1)(columns – 1) = 1
- Critical value at the 5% level, 1 degree of freedom: 3.841
- 97.4 is far beyond it, so the two are dependent. Keep the card
- Every expected count should be at least 5

These 500 customers are an invented sample, so the arithmetic fits on one slide.

3.5 Measuring Data for Classification

Correlation for two numbers, discretization for one

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$
$$= \frac{\text{cov}(x, y)}{(\sigma_x \cdot \sigma_y)} \quad -1 \leq r \leq +1$$

six customers: visits x, spend y (thousand VND)

x	y	x - $\bar{x}$	y - $\bar{y}$	product
2	200	-4	-400	1,600
3	280	-3	-320	960
5	380	-1	-220	220
6	620	0	+20	0
9	800	+3	+200	600
11	1,320	+5	+720	3,600

$$\bar{x} = 6, \quad \bar{y} = 600, \quad \sum \text{product} = 6,980$$

$$\sum (x - \bar{x})^2 = 60, \quad \sum (y - \bar{y})^2 = 869,600$$

$$r = 6,980 / \sqrt{60 \times 869,600} = 0.966$$

READING r

Near +1 the two rise together; near -1 one falls as the other rises; near 0 there is no linear relation, though a curve can hide there. r never proves cause.

Three ways to cut a numeric attribute

- Equal-width: a fixed step such as 50,000 VND. The skewed tail leaves many bins nearly empty
- Equal-frequency: every bin holds the same count. Three bins means cutting at the 33rd and 67th percentiles
- Quantile-based: cut at Q1 and Q3. This is what the basket band does, and it gives 25–50–25, not three equal bins
- Supervised: choose the cut point that maximizes information gain against the class

THE BASKET BAND IS QUANTILE-BASED

low < 70,500 · mid 70,500 to 138,500 · high > 138,500. Cut at Q1 and Q3, so the middle band holds half the rows.

3.6

Data Generalization and Summarization-Based Characterization

3.6 Generalization and Characterization

Concept description: the simplest kind of mining

Characterization

- Summarize the general features of one target class in a compact form
- Input: the task-relevant rows, for example every app order
- Output: a generalized relation, a crosstab, a chart, a characteristic rule
- Question answered: what do app orders look like?

Comparison

- Contrast the target class with one or more contrast classes
- Input: two sets of rows, for example repeat and one-time customers
- Output: the same forms, with a share for each class side by side
- Question answered: how do repeat customers differ from the rest?

3.6 Generalization and Characterization

Concept hierarchies: the ladder every attribute can climb



ONE ATTRIBUTE, SEVERAL LADDERS

store_name can climb by geography (district, city) or by business (store_type). Both are valid and they answer different questions. The ladder you pick is part of the analysis, exactly as the distance measure was in 3.4.

3.6 Generalization and Characterization

Two ways to generalize: data cube and attribute-oriented induction

Data cube approach (OLAP)

- Precompute aggregates for every combination of dimension levels
- Generalization is a roll-up along a hierarchy, and drill-down goes back
- Fast at query time, because the work was done at load time
- Limits: numeric measures only, and the hierarchies are fixed in advance

Attribute-oriented induction (AOI)

- Query-driven: collect the task-relevant rows first, then generalize them
- Works on any attribute type, with or without a cube
- The analyst sets thresholds and the algorithm decides how far to climb
- Output includes counts, t-weights and rules, and feeds class comparison

WHICH ONE THIS WEEK

Section 3.6 walks through AOI because it exposes every decision. When a warehouse with a cube exists, AOI runs its generalization as a roll-up and inherits its speed.

3.6 Generalization and Characterization

Attribute-oriented induction in four steps

1 Focusing

Collect the task-relevant rows with a query, for example every order line where channel = app. The result is the initial working relation.

2 Removal

Drop an attribute that has many distinct values and no hierarchy (order_id), or one a higher-level attribute already covers (store_name once store_type is kept).

3 Generalization

When an attribute has many distinct values and a hierarchy exists, replace values by the next level up. Repeat until the distinct count fits the attribute threshold.

4 Aggregation

Merge identical generalized tuples and accumulate a count, plus any sums or averages. Stop when the prime relation fits the relation threshold.

THE TWO THRESHOLDS

Attribute generalization threshold: the largest number of distinct values allowed for one attribute, usually 2 to 8.
Generalized relation threshold: the largest number of tuples allowed in the result. Both are set by the analyst, and both belong in the report.

3.6 Generalization and Characterization

Worked example: eight app order lines become four rows

order_id	order_time	store_name	product_name	line_total
50231	07:42	Nguyen Hue	Iced milk coffee	45,000
50235	08:05	Ben Thanh	Espresso	40,000
50240	08:50	Landmark 81	Cold brew	55,000
50244	08:10	RMIT	Iced milk coffee	45,000
50251	09:30	University Village	Peach tea	50,000
50302	12:20	Nguyen Hue	Croissant	35,000
50310	13:05	Ben Thanh	Cheesecake	65,000
50318	12:40	Ben Thanh	Croissant	35,000

Initial working relation: channel = app

daypart	store_type	category	count	t-weight
morning	mall	coffee	3	37.5%
midday	mall	pastry	3	37.5%
morning	campus	coffee	1	12.5%
morning	campus	tea	1	12.5%

Prime generalized relation, thresholds: attribute 4, relation 4

T-WEIGHT OF A TUPLE

The count of the tuple divided by the total count of the class: $3/8 = 37.5\%$.

WHAT EACH COLUMN DID

order_id: removed, a key with no hierarchy. order_time: generalized to daypart (morning 6 to 11, midday 11 to 14). store_name: generalized to store_type. product_name: generalized to category. line_total: removed here to keep the example small; in the lab it becomes a basket band or an average per tuple.

3.6 Generalization and Characterization

Efficient implementation: two scans and a hash table

Scan 1: collect statistics

- For each attribute, count the distinct values
- Decide: keep, remove, or generalize, and to which level
- Build the mapping value → generalized value from the hierarchy

Scan 2: generalize and merge

- Replace each value through the mapping, tuple by tuple
- Insert the generalized tuple into a hash table keyed on its values
- On a hit add 1 to the count, on a miss insert it with count 1

```
work = lines[lines.channel == 'app']          # 1 focus
work = work.drop(columns=['order_id'])        # 2 remove
work['daypart'] = to_daypart(work.order_time) # 3 gen
work['store_type'] = work.store_name.map(store_type)
work['category'] = work.product_name.map(category)
prime = (work.groupby(['daypart', 'store_type', 'category'])
         .size().rename('count').reset_index()) # 4 agg
prime['t_weight'] = prime['count'] / prime['count'].sum()
```

COST

Both scans are linear in the number of rows, and the hash table is bounded by the relation threshold, so it fits in memory.

3.6 Generalization and Characterization

Presenting the result: relation, crosstab, chart and rule

Four forms of the same summary

- Generalized relation: the table on the previous slide, with counts and t-weights
- Crosstab: daypart down the side, category across the top
- Chart: bar chart of t-weights, or a pie when there are few tuples
- Characteristic rule: the same relation written as logic

READ THE RULE FOR A MANAGER

Three quarters of app orders come from mall stores, split evenly between morning coffee and midday pastry. Campus app orders are a morning habit. A marketing team can act on that sentence.

t-weight of tuple q_a in the target class:

$$t_weight(q_a) = \text{count}(q_a) / \sum_i \text{count}(q_i)$$

Characteristic rule for app orders, from the prime relation opposite:

```
∀X, app_order(X) ⇒  
    (daypart = morning ∧ store_type = mall  
      ∧ category = coffee) [t: 37.5%]  
  ∨ (daypart = midday ∧ store_type = mall  
      ∧ category = pastry) [t: 37.5%]  
  ∨ (daypart = morning ∧ store_type = campus  
      ∧ category = coffee) [t: 12.5%]  
  ∨ (daypart = morning ∧ store_type = campus  
      ∧ category = tea) [t: 12.5%]
```

3.7

Analytical Characterization and Mining Class Comparisons

3.7 Characterization and Comparison

Analytical characterization: let the data choose the attributes

- 1 **Collect.** The target class rows, plus a contrast class when comparing
- 2 **Preliminary AOI.** Generalize to a candidate relation so each attribute has a handful of values
- 3 **Measure relevance.** Information gain, chi-square or correlation from 3.5, one number per attribute
- 4 **Filter.** Drop the attributes below a relevance threshold and keep the rest
- 5 **Final AOI.** Generalize the filtered relation and present it as a relation, a chart and a rule

ON THE FOURTEEN CUSTOMERS

With a threshold of 0.05, channel and loyalty_card stay while store_type and basket_band go.

WHY PLAIN AOI IS NOT ENOUGH

Plain AOI keeps whatever the analyst listed. Measuring first gives a shorter rule with higher t-weights.

3.7 Characterization and Comparison

Class comparison: target against contrast in five steps

- 1 **Collect.** Query the target class (repeat customers) and the contrast class (one-time customers)
- 2 **Relevance.** Run the 3.5 measures on the union and keep the attributes that separate the classes
- 3 **Synchronous AOI.** Generalize both classes to the same level. Same ladder, same rung
- 4 **Present.** A table with a count and a share for each class side by side, with t-weights and d-weights
- 5 **Visualize.** Bar charts of the shares per class, and discriminant rules with d-weights

THE STEP PEOPLE SKIP

If the target is generalized to daypart and the contrast to hour, the comparison table cannot be built. Generalize both classes to the same level of every hierarchy before comparing. That is what synchronous means.

3.7 Characterization and Comparison

Worked example: repeat customers against one-time customers

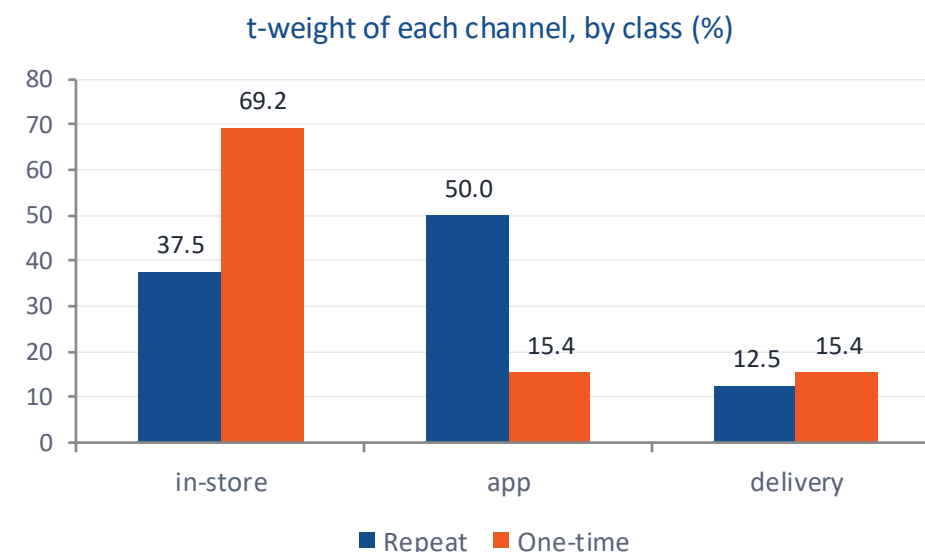
Main channel	Repeat (n = 240)	t-weight	One-time (n = 260)	t-weight	d-weight, repeat
in-store	90	37.5%	180	69.2%	33.3%
app	120	50.0%	40	15.4%	75.0%
delivery	30	12.5%	40	15.4%	42.9%

SAME TEACHING SAMPLE AS SLIDE 10

Invented counts, chosen so the arithmetic is clean. Chi-square on this 3 by 2 table is 70.7 with 2 degrees of freedom, against a critical value of 5.991, so channel and class are dependent.

TWO READINGS OF THE SAME ROW

Half of all repeat customers use the app (t-weight 50.0%), and three quarters of app users are repeat customers (d-weight 75.0%). In-store is common among repeat customers (37.5%), yet it tells you little, because two thirds of in-store customers never come back.



3.7 Characterization and Comparison

t-weight and d-weight: two questions, two denominators

```
t-weight    typicality, inside one class
  t_w(q, C) = count(q in C) /  $\Sigma$  count(C)
  "Of repeat customers, what share use the app?"
  t_w(app, repeat) = 120 / 240 = 50.0%

d-weight    discrimination, across classes
  d_w(q, C) = count(q in C) /  $\Sigma$  count(q)
  "Of app users, what share are repeat customers?"
  d_w(app, repeat) = 120 / (120 + 40) = 75.0%
```

Criterion	Characterization	Comparison
Classes	One target class	Target and at least one contrast class
Goal	Summarize the common features	Find what differs between classes
Question	What are repeat customers like?	How do repeat customers differ from one-time customers?
Measure	t-weight	d-weight

A HIGH T-WEIGHT CAN HIDE A LOW D-WEIGHT

In-store is the second most typical channel for repeat customers (t-weight 37.5%), and only a third of in-store customers are repeat customers (d-weight 33.3%). Typical is about the class. Discriminating is about the value. Report both, and say which question each answers.

3.7 Characterization and Comparison

From tables to rules: characteristic and discriminant rules

```
Characteristic rule (target class only, t-weights)
  ∀X, repeat(X) ⇒ channel(X) = app      [t: 50.0%]
      ∨ channel(X) = in-store [t: 37.5%]
      ∨ channel(X) = delivery [t: 12.5%]
```

```
Discriminant rule (the other way, d-weights)
  ∀X, channel(X) = app ⇒ repeat(X) [d: 75.0%]
  ∀X, channel(X) = in-store ⇒ repeat(X) [d: 33.3%]
```

```
Quantitative rule (both weights on one line)
  channel = app : repeat [t: 50.0%, d: 75.0%]
```

What a manager does with each

- Characteristic: describes the segment. Use it to design the loyalty programme around app users
- Discriminant: predicts membership. Use it to decide who receives the retention offer
- A discriminant rule with d-weight 75% is a classification rule with 75% confidence, which is where Chapter 4 begins

THE BANK VERSION OF THE SAME IDEA

Characterization: good borrowers are mostly aged 30 to 50 with income above 20 million a month. Comparison: 85% of good borrowers keep debt-to-income below 30%, while 70% of defaulters sit above 50%.

3.8

Association Rules, Correlation Analysis and Constraint-Based Mining

3.8 Association Rules and Constraints

Market basket analysis: receipts as transactions

Receipt	Items on the receipt
R1	coffee, croissant
R2	coffee, tea
R3	coffee, croissant, cheesecake
R4	tea, croissant
R5	coffee, croissant
R6	coffee, tea, croissant
R7	tea
R8	coffee, croissant, tumbler
R9	coffee, cheesecake
R10	tea, croissant, cheesecake

itemset any set of items, e.g. {coffee, croissant}
k-itemset an itemset with k items
transaction one receipt: its id and its products

$\text{support}(A \Rightarrow B) = P(A \cup B)$ receipts holding both
 $\text{confidence}(A \Rightarrow B) = P(B \mid A) = \frac{\text{sup}(A \cup B)}{\text{sup}(A)}$

coffee $\Rightarrow$ croissant sup 5/10 = 50% conf 5/7 = 71%
tea $\Rightarrow$ croissant sup 3/10 = 30% conf 3/5 = 60%

IN THE SAIGON BEAN FILE

A transaction is one order_id, and its items are the distinct products on that receipt. 51,108 receipts and 17 products make a small basket matrix. Real retail has tens of thousands of items, which is why the next slide matters.

3.8 Association Rules and Constraints

Frequent itemsets and the Apriori property

Level	Candidates with support, minsup = 3 receipts (30%)	Frequent
C1 → L1	coffee 7, tea 5, croissant 7, cheesecake 3, tumbler 1	coffee, tea, croissant, cheesecake
C2 → L2	{coffee, tea} 2, {coffee, croissant} 5, {coffee, cheesecake} 2, {tea, croissant} 3, {tea, cheesecake} 1, {croissant, cheesecake} 2	{coffee, croissant}, {tea, croissant}
C3 → L3	{coffee, tea, croissant} is pruned before counting, because its subset {coffee, tea} is not frequent	none

The Apriori property

- Every non-empty subset of a frequent itemset is also frequent
- Read backwards: if an itemset is infrequent, every superset is infrequent too
- Size-k candidates are joined from frequent sets of size k – 1, and any candidate with an infrequent subset is dropped before counting
- The property is anti-monotone, the same idea that constraints use later

WHY IT SAVES WORK

With 17 products there are 131,071 possible itemsets. Apriori counted 5 singles and 6 pairs and never counted a triple. On the full file the pruning is what makes the search finish, and the two frequent pairs are exactly the itemsets a rule can be built from.

3.8 Association Rules and Constraints

From frequent itemsets to strong rules

```
For each frequent itemset L and each non-empty  $s \subset L$ :  
  rule  $s \Rightarrow (L - s)$   
  keep it when  $\text{support}(L) / \text{support}(s) \geq \text{minconf}$ 
```

```
L = {coffee, croissant}, support 50%  
  coffee  $\Rightarrow$  croissant      5/7 = 71%   strong  
  croissant  $\Rightarrow$  coffee      5/7 = 71%   strong
```

```
L = {tea, croissant}, support 30%  
  tea  $\Rightarrow$  croissant         3/5 = 60%   barely strong  
  croissant  $\Rightarrow$  tea         3/7 = 43%   dropped
```

```
minsup = 30%, minconf = 60%
```

Reading a rule

- Support says how often the rule applies; 0.5% of 51,108 receipts can still be worth money
- Confidence says how reliable it is once the antecedent is on the receipt
- The same itemset yields several rules with different confidence, so direction matters
- Strong means both thresholds are cleared. Strong is not the same as interesting

THRESHOLDS ARE BUSINESS DECISIONS

A supermarket with a million receipts sets minsup at 0.1%; a coffee chain with 17 products can afford 2%. Report both thresholds with every rule.

3.8 Association Rules and Constraints

Strong is not the same as interesting: lift and chi-square

$$\begin{aligned}\text{lift}(A \Rightarrow B) &= P(A \cup B) / (P(A) \cdot P(B)) \\ &= \text{confidence}(A \Rightarrow B) / P(B)\end{aligned}$$

> 1 together more than chance: positive
= 1 independent: the rule adds nothing
< 1 A discourages B: negative

tea $\Rightarrow$ croissant confidence 60% P(croissant) 70%
lift = $0.30 / (0.50 \times 0.70) = 0.86$

10,000 receipts	coffee	no coffee	total
croissant	4,000	2,000	6,000
no croissant	3,500	500	4,000
total	7,500	2,500	10,000

CROISSANT $\Rightarrow$ COFFEE

support 40%, confidence 66.7%, P(coffee) = 75%. lift = $0.667 / 0.75 = 0.89$. Strong under minconf 60%, and still negative.

THE SAME TEST AS 3.5

Expected count for (croissant, coffee) is $6,000 \times 7,500 / 10,000 = 4,500$ against 4,000 observed. $\chi^2 = 555.6$, far above 3.841, and the observed count sits below the expected one, so the dependence is negative. Lift and chi-square agree; confidence alone would have recommended bundling the two.

3.8 Association Rules and Constraints

Constraint-based mining: tell the algorithm what you care about

Kind of constraint	What it restricts	Saigon Bean example
Knowledge type	Which kind of pattern to mine	Association rules, not classification or clustering
Data	Which rows to mine	Receipts from campus stores, June to August 2026
Dimension or level	Which attributes, and which rung of the hierarchy	Products at the category level, or at product_name level
Interestingness	Thresholds on the measures	minsup 2%, minconf 50%, lift above 1.2
Rule or pattern	The form of the rule	The antecedent contains a coffee and the consequent a pastry, with a total list price of at most 200,000 VND

PUSH THE CONSTRAINT INTO THE SEARCH

Anti-monotone: break it once and every superset breaks it, so the search prunes as Apriori does

Monotone: satisfy it once and it stays satisfied

Succinct: candidates come straight from the product table

WHY ANALYSTS NEED IT

Unconstrained mining on 51,108 receipts returns thousands of rules, most of them variations of one another. Constraints turn a listing into an answer to a question.

Summary

- 1 A class label turns the question around: 3.4 measured the distance between rows, 3.5 measures what a column tells you about the label.
- 2 Entropy is the uncertainty of a label in bits, and information gain is how much of it an attribute removes. Attributes with many values are favoured.
- 3 Chi-square tests whether a nominal attribute and the class are independent. Correlation does the same job for two numeric attributes.
- 4 Concept description climbs concept hierarchies until rows collapse into a few with counts. Data cubes roll up, AOI generalizes query by query.
- 5 AOI runs in two linear scans: one to collect statistics and choose the level, one to generalize and merge through a hash table.
- 6 Class comparison generalizes target and contrast to the same level. t-weight is typicality inside a class, d-weight is separation across classes.
- 7 Association rules are judged by support and confidence, found by Apriori, which prunes every superset of an infrequent set, then checked with lift.
- 8 Constraints on data, level, thresholds and rule form turn thousands of rules into an answer, and anti-monotone ones prune the search itself.



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KINH TẾ - LUẬT

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF ECONOMICS AND LAW

THANK YOU!

Next: Chapter 4. Classification.

